Clase 5 — El retardo en las redes de paquetes

Bloque: A (Fundamentos) · **Semana:** 2 · **Duración estimada:** 110 min **Correlativa:** Clase 4 (conmutación: circuitos, paquetes y mensajes) · **Sigue con:** Clase 6 (taller de retardos, todo resuelto) **Material de referencia:** Tanenbaum cap. 1 §1.5 y §1.2 (pág. ~10 a ~30 del PDF de tu carpeta) · RFC 9293, RFC 5681, RFC 5437, RFC 9000

🎯 Al terminar esta clase vas a poder

1. **Escribir** el retardo total como **suma de cuatro componentes**, con la fórmula, la unidad y de qué depende cada uno.
2. **Distinguir** `d_transmisión = L/R` de `d_propagación = d/s`, y explicar por qué el primero **no depende del medio** y el segundo **no depende del paquete**.
3. **Decir** cuál es el **único componente variable** y **el único que refleja la congestión**, y explicar la cadena congestión → retardo de cola → retardo total.
4. **Calcular** el retardo de un paquete en un enlace y en un camino de varios enlaces, con la aritmética escrita paso a paso.
5. **Calcular** el *bandwidth-delay product* (`BDP = R · RTT`) y explicar por qué una conexión de larga distancia necesita una ventana grande.
6. **Distinguir** latencia de RTT, y **calcular** el jitter de un flujo.
7. **Definir** congestión, pérdida y retardo, y explicar el vínculo entre las tres, como lo pide el Examen 2011-03.
8. **Resolver entero** el Examen `2009-12 P1` (10 pts) con la solución oficial a la vista.

> **Prerequisito del examen:** `2009-12 P1` (10 pts — componentes del retardo, ancho de banda vs. velocidad de propagación, congestión), `2011-03 P2` (6 pts — congestión, pérdidas y retardo), `2011-03 P5` (10 pts — retardo end-to-end → **Clase 6**), `2010-02 P5` (jitter vs. retardo), `2009-03 P5` (full/half duplex).

PARTE 1 — TEORÍA

1.1 El retardo total: la ecuación

Todo lo que aprendiste hasta ahora se resume en **una sola suma**. Cuando un paquete viaja de A a B atravesando `n` enlaces y `k` routers:

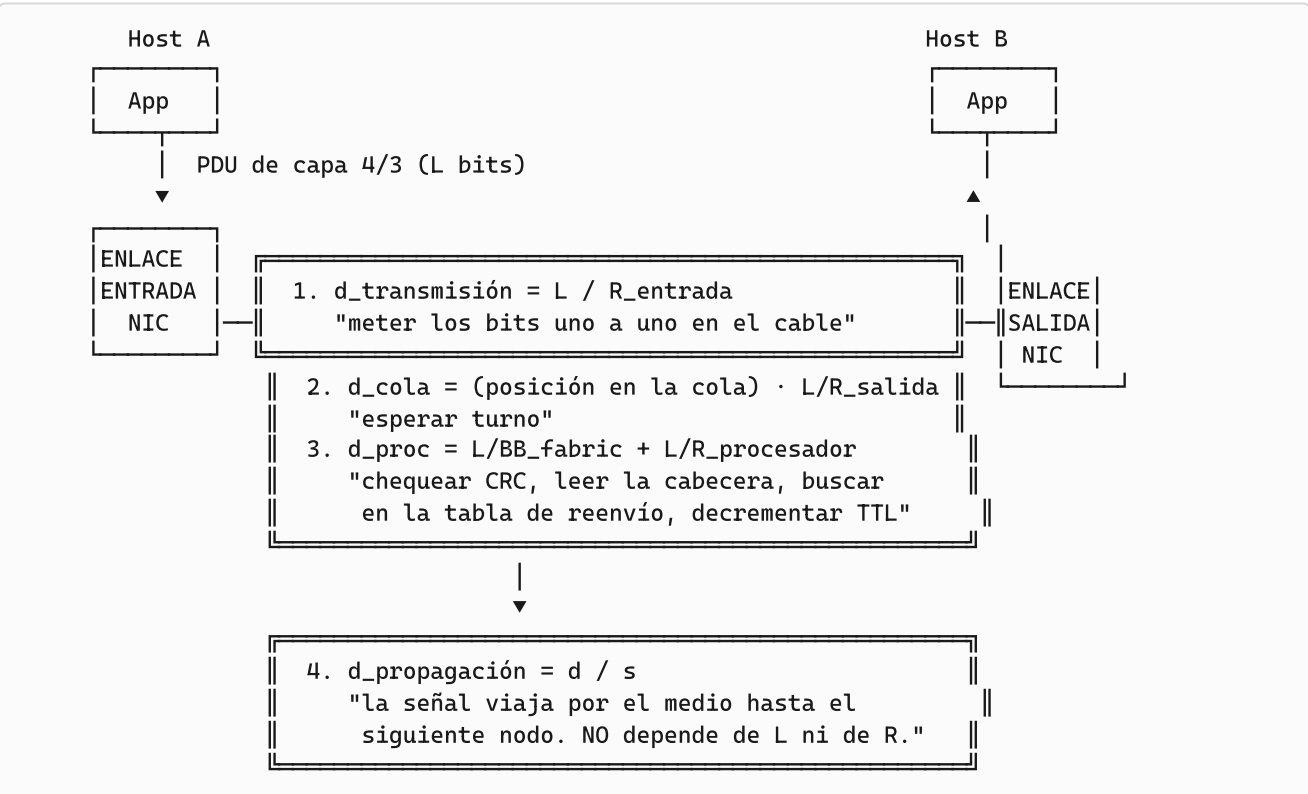
```
╔═══════════════════════════════════════════════════════════╗
║                                                           ║
║   d_end2end  =   Σ  ( d_proc(i)  +  d_cola(i)  +          ║
║                     d_transmisión(i)  +  d_propagación(i) )  ║
║                                                           ║
║                i = 1 ... n                                ║
║                                                           ║
╚═══════════════════════════════════════════════════════════╝
```

Y **el detalle que se pregunta siempre**: hay `n` **términos de propagación y** `n` **de transmisión** (uno por enlace), pero hay `k` **términos de procesamiento** (uno por router, y un router **no es** un enlace).

```
n enlaces ──────►  n · d_transmisión  +  n · d_propagación
k routers  ──────►  k · d_proc         +  k · d_cola
(con k = n − 1 en una línea sin ramales)

⚠️ EL ERROR TÍPICO ES PONER n · d_proc EN VEZ DE k · d_proc.
   El Examen 2011-03 P5 lo pide explícitamente y es la
   pregunta más repetida del curso.
```

El diagrama de un solo router, que es donde se ve cada componente:



1.2 Los cuatro componentes, uno por uno

1.2.1 Retardo de procesamiento (`d_proc`)

Qué es. El tiempo que tarda el nodo en **hacer el procesamiento** del paquete entre que llega por una interfaz y sale por otra: leer la cabecera, verificar el CRC, decrementar el TTL, recalcular el checksum, consultar la tabla de reenvío (*longest prefix match*), y en un switch, aprender la dirección MAC origen.

Fórmula (en un router real). Un modelo desagregado en hardware separa el ancho de banda del **fabric**:

```
d_proc = d_cola_ingreso  +  L / B_fabric  +  d_bus  +  L / R_procesador
           └ en la interfaz ┘   └ el "backplane" ┘  └ el bus de la CPU┘

Con un fabric de 40 Gbps, un bus de memoria de 33 MHz y un procesador
de 1 GHz:

   L = 1500 bits
   L / B_fabric      = 1500 / 40·10⁹ = 37,5 ns
   L / R_procesador  = 1500 / 10⁹    = 1 500 ns = 1,5 µs
   bus + otros                ≈       100 ns

   d_proc ≈ 1,64 µs
```

Interpretación — y por qué se lo considera despreciable.

```
Comparación en un enlace de 1 Gbps con un paquete de 1500 bytes
y 500 km de fibra:

   d_transmisión = 1500 bits / 10⁹ = 1,5 µs
   d_propagación = 500 000 m / 2·10⁸ = 2,5 ms = 2 500 µs
   d_proc                              ≈ 1,6 µs
   ──────────────────────────────────────────────────────
   PESO RELATIVO:
      d_proc        =  1,6 µs  →   0,06 %  del total
      d_transmisión =  1,5 µs  →   0,06 %  del total
      d_propagación =  2 500 µs →  99,9 %   del total
```

Tres razones por las que se lo desprecia:

1. **Es tres órdenes de magnitud menor** que la propagación en cualquier enlace de backbone.
2. **Es independiente del tráfico.** No depende de cuántos paquetes hay ni de la carga: el router procesa un paquete por vez en la CPU. Por eso **no se ve afectado por la congestión**.
3. **Los equipos modernos lo hacen en hardware.** Un *lookup* en TCAM es determinista y de ~40 ns. Los conmutadores de 400 Gbps procesan **billones de paquetes por segundo**: si el procesamiento fuera de 1 µs por paquete, un solo puerto necesitaría 10 Gbps solo de CPU.

Cuándo SÍ importa. En redes pequeñas, con equipos viejos, o cuando hay **procesamiento de software** encima:

- Un **firewall que inspecciona la carga útil** (un WAF, un IDS de contenido) puede tardar **decenas de microsegundos** por paquete.
- Un **router doméstico con NAT y registro de conexiones** puede tardar **1 a 5 µs**.
- Una **VPN con cifrado por software** puede tardar **5 a 20 µs** (y por eso las implementaciones usan cifrado por hardware: las instrucciones AES-NI y las claves de 10 Gbps).

En un problema de examen, salvo que el enunciado lo diga explícitamente, **se desprecia**. Pero **decirlo** suma: "desprecio `d_proc` porque es de microsegundos contra milisegundos de propagación, y porque no depende de la carga".

1.2.2 Retardo de encolamiento (`d_cola`)

Qué es. El tiempo que el paquete **espera en el buffer** antes de que le toque ser transmitido.

Fórmula. Si el paquete es el `k`-ésimo de una cola de paquetes de `L` bits cada uno:

```
d_cola = k · L / R_salida
```

Donde `R_salida` es la velocidad del enlace **de salida** (¡no la de entrada!).

El diagrama de la cola, que es donde se ve por qué es el único variable:

```
ENLACE DE SALIDA, 10 Mbps (un paquete de 1000 bits cada 0,8 ms)

tiempo ──────────────────────────────────────────────────────►
0        0,8      1,6      2,4      3,2      4,0        4,8 ms
|        |        |        |        |        |          |
▼        ▼        ▼        ▼        ▼        ▼          ▼
┌──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┐
│  P1  │  P2  │  P3  │  P4  │  P5  │  P6  │  P7  │  P8  │      │      │      │      │ ← cola
└──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┘
                                                  ↑
                                          B = fin del buffer

d_cola(P1) = 0
d_cola(P2) = 0,8 ms
d_cola(P3) = 1,6 ms
...
d_cola(Pk) = (k-1) · 0,8 ms

⚠️ Si el paquete número 100 llega cuando hay 8 paquetes en el
   buffer, y la cabecera del buffer es 10 paquetes: el paquete 9
   y el 10 se DESCARTAN (tail drop).
```

Las cuatro propiedades de `d_cola`

Propiedad 1 — Es el ÚNICO componente variable. Los otros tres se pueden calcular con los datos del problema; `d_cola` no, porque depende del estado de la cola, que es información que el enunciado normalmente **no da**.

Componente	¿Se puede calcular con los datos del enunciado?	¿Depende de la carga?
`d_proc`	Sí, si te lo dan	**No**
`d_cola`	**No**	**Sí**
`d_transmisión`	Sí (`L` y `R`)	**No**
`d_propagación`	Sí (`d` y `s`)	**No**

Propiedad 2 — Es el ÚNICO que refleja la congestión. Esta es la respuesta literal de la **Pregunta 1b del Examen 2009-12** (que se resuelve entera en el §3.2).

```
MÁS TRÁFICO OFRECIDO
      │
      ├──→ las colas se LLENAN más
      │           │
      │           ├──→  d_cola  AUMENTA  ──→  retardo total AUMENTA
      │           │
      │           └──→  si se supera la capacidad del buffer:
      │                       se DESCARTA  ──→  PÉRDIDAS
      │
      └──→ se />}
```

Propiedad 3 — Puede ser CERO. Si el paquete llega y el enlace de salida está libre, `d_cola = 0`. Por eso el enunciado del Examen 2011-03 P5 dice explícitamente "*suponiendo que no se produce retardo de cola*": es una hipótesis, y es la que hace el problema resoluble.

Propiedad 4 — Es el único componente que NO tiene cota superior. `d_proc`, `d_trans` y `d_prop` están determinados por el hardware y por la física. `d_cola` puede valer 1 ms o 5 segundos, según lo saturado que esté el enlace. **Ese es el motivo por el que el retardo en Internet no tiene garantía: porque un componente del total es inacotado.**

La pérdida, que es el otro extremo de `d_cola`

Cuando la cola está llena y llega un paquete más, **se descarta** (*tail drop*). Y hay que entender por qué **agrandar el buffer no es la solución**:

```
Llega más tráfico del que sale:
   tasa de llegada  = 10 Gbps
   tasa de salida   = 100 Mbps

Se acumulan 9,9 Gbps = 1,2375 GB por segundo

Buffer = 4 MB  → se llena en 4·10⁶ / 1,2375·10⁹ = 0,0032 s
Buffer = 400 MB → se llena en 0,32 s
Buffer = 4 GB   → se llena en 3,2 s

⚠️ El buffer agrandado retrasa la muerte, no la evita.
   Y además INCREMENTA el retardo de los que sí pasan:
   un paquete que espera 3,2 s en un buffer de 4 GB tiene un
   retardo de 3,2 s. Eso es BUFFERBLOAT.
```

La solución real: los AQMs (*Active Queue Management*). En vez de esperar a llenar el buffer, el router **descarta (o marca) temprano**. **CoDel** (*Controlled Delay*) descarta cuando el retardo de un paquete

supera un objetivo de 5 ms durante un intervalo; **FQ-CoDel** es el que usa Linux por defecto desde el kernel 3.5 (2012); **PIE** es otro. Y **ECN** (`RFC 3168`) marca en vez de descartar. Todo esto es de la Clase 21.

1.2.3 Retardo de transmisión (`d_transmisión`)

Qué es. El tiempo que tarda el emisor en **meter el paquete entero en el cable**. Es decir, desde que empieza a salir el primer bit hasta que sale el último.

```
──────────────────────────────────────────────────────────────────
 ENLACE de 10 Mbps, paquete de 1500 bits:

 bit:  0   1   2   3   4   5   6  ...  1498  1499  ← 1500 bits
       |   |   |   |   |   |   |        |     |
 t(s): 0   1  2  3  4  5  6      149,8   150   × 10⁻⁷ s
                                          ↑
                                  el último bit sale a los 150 µs

 d_transmisión = 1500 bits / 10·10⁶ bit/s = 150 µs = 0,15 ms
──────────────────────────────────────────────────────────────────
```

La fórmula:

```
┌──────────────────────────────────┐
│         L      (en BITS)         │
│ d_t = ─────                      │
│         R   (en bps)             │
└──────────────────────────────────┘

⚠️ L EN BITS. Si te dan el paquete en BYTES, multiplicá por 8.
   1500 bytes = 12 000 bits.  Si hacés 1500/10⁷ da 0,15 ms, que
   es 8 veces MENOR que la respuesta correcta: ese es el error
   número uno de los ejercicios de retardo.
```

Las dos propiedades que hay que decir siempre:

1. **No depende del medio.** El medio (cobre, fibra, aire) tiene la misma `R`. La `R` depende del **equipo**: del transceptor y de la modulación.
2. **No depende de la distancia.** `d_trans` es el mismo para un enlace de 1 m y uno de 1000 km, si el ancho de banda es el mismo. Es un retardo **local**, no de la señal.

Un caso límite que hay que ver. Con un paquete de **1 bit** y un enlace de 1 Gbps:

```
d_trans = 1 / 10⁹ = 1 ns
```

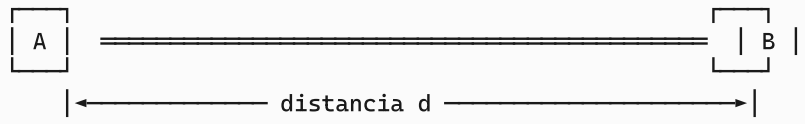
Retardo de transmisión despreciable frente a la propagación en redes largas:

```
5000 km de fibra, 10 Gbps, paquete de 1500 bytes:
   d_trans = 12 000 / 10¹⁰ = 1,2 µs
   d_prop  = 5·10⁶ / 2·10⁸ = 25 ms
   ──────────────────────────────────────
   d_prop / d_trans = 25 000 / 1,2 = 20 833 veces
```

Es el mismo cálculo del §1.2.1, y da la misma conclusión: **en redes de backbone, todo el retardo es propagación**.

1.2.4 Retardo de propagación (`d_propagación`)

Qué es. El tiempo que tarda **la señal física** en recorrer el medio, desde el momento en que sale el último bit de la interfaz del emisor hasta que llega el primero a la interfaz del receptor.




```
El último bit sale de A en t = 0.
Ese bit viaja por el medio y llega a B en t = d/s.
⚠️ OJO: el "primer bit" ya había llegado antes, hace un
   d_trans. Lo que se mide acá es la propagation DEL ÚLTIMO BIT.
```

La fórmula:

$$d_p = \frac{d}{s}$$

d (en METROS) / s (en m/s)

Los valores de `s` que hay que saber de memoria:

MEDIO	s	d para 1000 km
VACÍO (espacio libre)	$3 \cdot 10^8$ m/s	3,33 ms
AIRE	$2{,}998 \cdot 10^8$ m/s	3,34 ms
COBRE (UTP/STP, coaxial)	$\sim 2 \cdot 10^8$ m/s	5,00 ms
FIBRA ÓPTICA (silicio)	$\sim 2 \cdot 10^8$ m/s	5,00 ms
POLVO / GAS (nuclear)	$\sim 3 \cdot 10^8$ m/s	3,33 ms

```
Para problemas de examen con cobre o fibra:  s = 2 · 10⁸ m/s
Para un enlace por satélite:                 s = 3 · 10⁸ m/s
```

Las dos propiedades:

1. **No depende del paquete.** Un paquete de 40 bits y uno de 15 000 bits tardan **lo mismo** en propagarse por el mismo enlace. La señal es la misma.
2. **No depende del equipo ni del ancho de banda.** Cambiar de 1 Mbps a 100 Mbps **no cambia** `d_prop`. Cambiar de cobre a fibra tampoco (2,00 → 2,04 · 10^8, un 2 %).

Y la diferencia con la velocidad de propagación, que es el contenido del Examen 2009-12 P1a:

```
s  NO es un ancho de banda.  Está en m/s, no en bps.
s  NO es "la velocidad del enlace". La velocidad del enlace es R.
s  es una propiedad del MEDIO FÍSICO.
```

1.2.5 Las dos tablas comparativas

Tabla A — De qué depende cada componente

	d_proc	d_cola	d_trans	d_propagación
¿Depende de L?	NO	NO	SÍ	NO
¿Depende de R?	NO	SÍ	SÍ	NO
¿Depende de d?	NO	NO	NO	SÍ
¿Depende de s?	NO	NO	NO	SÍ
¿Depende del medio?	NO	SÍ	NO	NO
¿Depende de la carga/congestión	NO	SÍ	NO	NO
¿Se puede calcular con los datos?	si te lo dan	NO	SÍ	SÍ
¿Cuota con R?	NO	NO	SÍ	NO
¿Cuota con s?	NO	NO	NO	SÍ
UNIDAD	s, ms	s, ms	s, ms	s, ms
TAMAÑO TÍPICO	~1 µs	0 a ∞	µs a ms	ms a s
¿Es VARIABLE?	NO	**SÍ**	NO	NO
¿Refleja la congestión?	NO	**SÍ**	NO	NO

La última fila es la que se pregunta en el examen. Y la penúltima: "señale cuáles son los más significativos" ⇒ **la propagación, porque es la única que siempre está y es la única que domina en enlaces largos**.

Tabla B — Cuánto pesa cada uno

```
ENLACE DE 1 Gbps, PAQUETE DE 1000 BYTES, DISTANCIA VARIABLE

L = 1000 bytes = 8000 bits
R = 10⁹ bit/s
d_trans = 8000/10⁹ = 8 µs
d_proc  ≈ 1 µs
s = 2·10⁸ m/s
```

Distancia	d_prop	d_total	Cuánto pesa d_prop
1 m	5 ns	9,005 µs	0,06 %
100 m	0,5 µs	9,5 µs	5,3 %
1 km	5 µs	14 µs	35,7 %
100 km	500 µs	509 µs	98,2 %
5 000 km	25 ms	25,009 ms	99,996 %
36 000 km (satélite)	180 ms	180,008 ms	99,996 %

La lectura de la tabla, que es lo que hay que decir:

- **En una LAN**, el retardo es **todo de procesamiento y transmisión**: a 1 metro, `d_prop` es el 0,06 %. Por eso un `ping` a un host en el mismo switch da **0,05 ms**, que es puramente `d_proc + d_trans`.
- **A 100 km**, `d_prop` ya es el 98 %. A partir de ahí, **duplicar la distancia duplica el retardo**, y mejorarla red significa **acortar la distancia**, no mejorar el equipo.
- **A 36 000 km (satélite)**, `d_prop` = **180 ms** de ida, **360 ms de RTT**. Ninguna mejora de hardware lo cambia: es geometría.

Conclusión operativa:

```
EN UNA LAN:  mejorar el retardo = mejorar el EQUIPO
EN UNA WAN:  mejorar el retardo = acortar el CAMINO
```

1.3 Ancho de banda vs. velocidad de propagación

Esta es la **Pregunta 1a del Examen 2009-12** y hay que tenerla respondida con las palabras exactas. Ya la vimos en la Clase 1; acá va la versión con la derivación matemática que la justifica.

La tabla

	Ancho de banda `B` o `R`	**Velocidad de propagación `s`**
Unidad	**bits por segundo (bps)**	**metros por segundo (m/s)**
Qué mide	Cuántos bits por segundo	A qué velocidad viaja la señal
De qué depende	Del **equipo**: puerto, transceptor, modulación	Del **medio físico**: la física del material
Depende de la distancia	No	No
Aparece en	`d_transmisión = L / R`	`d_propagación = d / s`
Ejemplo de enunciado	"el enlace tiene 1 Gbps"	"en ese cable la señal va a $2 \cdot 10^8$ m/s"

Por qué son perpendiculares: la demostración con números

Dos enlaces del mismo operador, a la misma distancia, con la misma fibra:

```
Enlace A: 1 Gbps    d = 1000 km
Enlace B: 100 Mbps  d = 1000 km
s = 2·10⁸ m/s en los dos (misma fibra)

d_prop(A) = 10⁶ / 2·10⁸ = 5,0 ms
d_prop(B) = 10⁶ / 2·10⁸ = 5,0 ms      ← IDÉNTICO

L = 12 000 bits
d_trans(A) = 12 000 / 10⁹  = 12 µs
d_trans(B) = 12 000 / 10⁸  = 120 µs

d_total(A) = 5,000012 ms
d_total(B) = 5,000120 ms

DIFERENCIA = 0,000108 ms = 108 ns

⚠️ 1000 veces más ancho de banda, y el retardo es el mismo
   salvo en 108 nanosegundos.
```

Y el otro lado de la demostración, que es la que convence: mismo ancho de banda, distinto medio.

```
Enlace C: 1 Gbps por FIBRA
Enlace D: 1 Gbps por COBRE
R = 10⁹ en los dos

d_trans(C) = 12 µs
d_trans(D) = 12 µs          ← IDÉNTICO

d_prop(C) = 10⁶ / 2,04·10⁸ = 4,90 ms
d_prop(D) = 10⁶ / 2,00·10⁸ = 5,00 ms

DIFERENCIA = 0,10 ms = 100 µs

⚠️ La fibra es 1,1 % más rápida en propagación, pero gana
   en otro lado: atenuación mucho menor (26 dB/km contra 20),
   por lo que alcanza 100 km sin repetidor contra los 100 m
   del cobre.
```

La frase de cierre para el examen:

> El ancho de banda y la velocidad de propagación **no son comparables**: uno mide una cantidad de información por unidad de tiempo (`bps`) y el otro una rapidez física (`m/s`). El primero depende del **equipo** y aparece en el retardo de transmisión; el segundo depende del **medio** y aparece en el retardo de propagación. **Duplicar el ancho de banda reduce el retardo de transmisión a la mitad y no cambia el de propagación.** Un enlace de 1 Gbps no es "50 veces más rápido" que uno de 100 Mbps: es 50 veces más **ancho**.

1.4 El *bandwidth-delay product*: los bits que están en vuelo

La definición

```
┌──────────────────────────────────────────────┐
│                                              │
│   BDP = R · RTT        [bits]                │
│                                              │
│   = ancho de banda (bps) × tiempo de ida y   │
│     vuelta (s)                               │
│                                              │
│   = los BITS que caben en el enlace durante un │
│     viaje completo de ida y vuelta           │
│                                              │
└──────────────────────────────────────────────┘
```

Por qué ese número importa

La idea es esta: **mientras un paquete va de ida, el emisor puede seguir mandando**. Si manda un paquete cada vez que llega un ACK, el único paquete "en vuelo" es el que está en el cable en ese instante. Si en cambio manda una **ventana** de `W` bytes sin esperar, el enlace lleva `W` bytes en tránsito al mismo tiempo.

```
El enlace es un CAÑO. El BDP es cuánta agua le entra.

┌──────────────────────────────────────────────────────────┐
│                                                          │
│   A ═══════════════════════════════════════ B            │
│                                                          │
│   ◄──────── L = 12 000 km ────────►                      │
│                                                          │
│   s = 2·10⁸ m/s   →  d/s = 12·10⁶/2·10⁸ = 60 ms de ida   │
│   R = 1 Gbps      →  10⁹ bits/s                          │
│                                                          │
│   BDP = 10⁹ × (2 × 60·10⁻³) = 10⁹ × 0,12 = 1,2·10⁸ bits  │
│                                    = 15 MB = 15 000 000 B│
│                                                          │
│   Para LLENAR el caño de punta a punta necesito 15 MB    │
│   "en vuelo" al mismo tiempo.                            │
└──────────────────────────────────────────────────────────┘
```

El cálculo, con la aritmética visible

Ejemplo: enlace de 1 Mbps entre Montevideo y Buenos Aires, 1 200 km de fibra, `s = 2·10⁸ m/s`.

```
d (de ida) = 1 200 km = 1,2·10⁶ m
s          = 2·10⁸ m/s

d_prop (ida) = 1,2·10⁶ / 2·10⁸ = 6·10⁻³ s = 6 ms
RTT ≈ 2 × d_prop = 12 ms   (ignorando cola y procesamiento)

R = 10⁶ bit/s

BDP = R × RTT = 10⁶ bit/s × 12·10⁻³ s
    = 12·10³ bits
    = 12 000 bits
    = 1 500 bytes        ← el BDP en BYTES
```

Interpretación: en ese enlace, en cada momento hay como máximo **1500 bytes** en tránsito. Si tu ventana de TCP es de **1024 bytes**, el enlace **nunca** se llena: mandás 1024 bytes, esperás el ACK, y el caño queda con 1024 bytes de un máximo posible de 1500.

```
Throughput máximo alcanzable = ventana / RTT

con ventana = 1024 B:  1024 / 0,012 s = 85 333 B/s = 0,68 Mbps
                                  ⚠️ contra 1 Mbps de enlace
con ventana = 1500 B:  1500 / 0,012 s = 125 000 B/s = 1,00 Mbps  ✓
con ventana = 4096 B:  4096 / 0,012 s = 341 333 B/s = 2,73 Mbps
                    ⚠️ más que el enlace: no se puede,
                       el enlace es el techo
```

La regla que sale de acá, y que es el motivo de `Window Scale`:

> Para aprovechar toda la capacidad de un enlace, la ventana de TCP tiene que ser **mayor o igual al BDP**. Si es menor, el throughput queda limitado por la ventana, no por el enlace.

Y por qué las conexiones de larga distancia necesitan ventanas enormes:

```
Enlace                R        RTT          BDP        ¿ventana de 64 KB alcanza?
──────────────────────────────────────────────────────────────────────────────
LAN 1 Gbps            1 Gbps   0,1 ms       12,5 kB    SÍ, de sobra
Metro 10 Gbps         10 Gbps  1 ms          1,25 MB    NO (64× corto)
Continental 10 Gbps   10 Gbps  120 ms        150 MB     NO (2400× corto)
Satélite 1 Gbps       1 Gbps   600 ms        75 MB      NO (1200× corto)
Satélite 100 Mbps     100 Mbps 600 ms        7,5 MB     NO (120× corto)
```

Y de ahí salen dos cosas concretas:

1. `Window Scale` (`RFC 7323`) multiplica la ventana por una potencia de 2 que se negocia en el SYN. Sin él, el máximo son 65 535 bytes, y **con los BDP de las redes de backbone de hoy eso es inútil**. Es un protocolo de 2014 que resolvió un problema de 1992, y sigue siendo obligatorio.
2. **Las conexiones de larga distancia seFAULTS de throughput** si no escalan la ventana: no por falta de ancho de banda, sino porque **no tienen suficiente "agua" en el caño**.

1.5 RTT, latencia y jitter

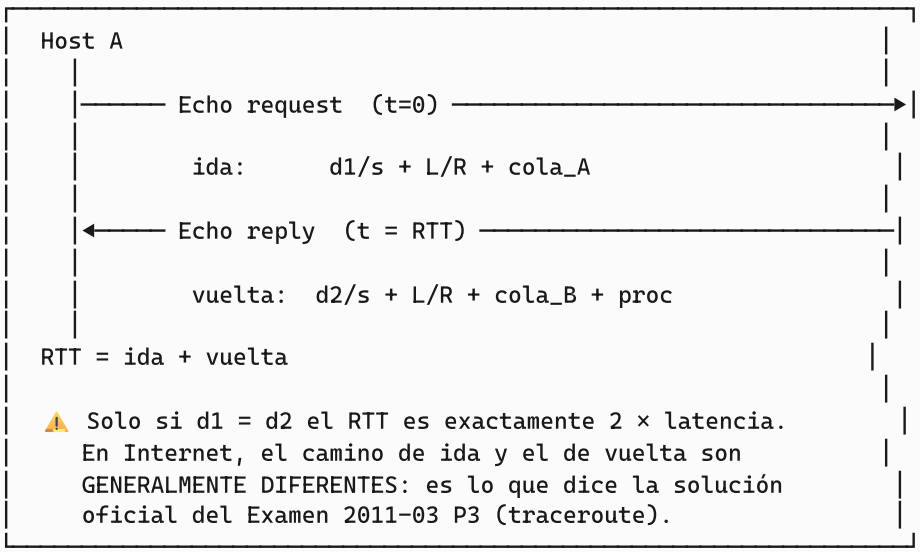
Latencia

Es el **tiempo de un viaje de ida**, de un host al otro. Se mide con el primer paquete de un `traceroute` (más `d_trans`) o con `ping` y la mitad del RTT.

```
latencia mínima ≈ d/s  +  L/R      (sin cola, sin procesamiento)
```

RTT (*Round Trip Time*)

Es el **tiempo de ida y vuelta**. Es la latencia multiplicada por 2, **más** el procesamiento de los dos extremos, **más** la cola si la hay.



Cómo se mide: `ping` mide el RTT porque manda un Echo request (ICMP tipo 8) y espera el Echo reply (ICMP tipo 0). Cada salida del comando da **un RTT**.

```
$ ping www.udelar.edu.uy
64 bytes from 203.0.113.9: icmp_seq=1 ttl=54 time=28,4 ms
64 bytes from 203.0.113.9: icmp_seq=2 ttl=54 time=28,1 ms
64 bytes from 203.0.113.9: icmp_seq=3 ttl=54 time=29,0 ms
```

Los tres valores son RTT. **La latencia de ida es ~14 ms.**

La primera diferencia de la serie es el RTT mínimo, y la siguiente es la variación (que es el jitter).

Jitter

Es **la variación de la latencia entre paquetes del mismo flujo**. Se mide de varias formas:

Métrica	Fórmula	Ventaja
Pico a pico (la que más se usa en la práctica)	`retardo_máx − retardo_mín`	Simple, la que usa el *jitter buffer*
Desviación estándar (RFC 3550)	`J = √( (1/n)·Σ (R_i − R_esperado)² )`	La que usa RTP
Media absoluta	`J = (1/n)·Σ |R_i − R_promedio|`	Intermedia

```
Tres paquetes del mismo flujo, enviados con 20 ms de separación:
   paquete 1:  80 ms
   paquete 2:  95 ms
   paquete 3: 110 ms

jitter (pico a pico) = 110 − 80 = 30 ms
promedio             = (80+95+110)/3 = 95 ms
desviación estándar  = √(((80−95)²+(95−95)²+(110−95)²)/3)
                     = √((225+0+225)/3) = √150 = 12,2 ms
```

Por qué el jitter importa más que la latencia (y por qué importa para audio y video)

Para un archivo, da igual: si tarda 100 ms o 400 ms, total ya esperaste.

Para una llamada de voz, la latencia alta se nota como un "eco del parque" (retraso de la voz), y el humano lo tolera hasta ~150 ms de ida. Lo que **rompe** la conversación es el jitter:

```
Un códec de voz reproduce una muestra cada 20 ms.

El paquete 1 tardó 80 ms → llega en t = 80
El paquete 2 tardó 110 ms → llega en t = 130
    ⚠️ Pero el reproductor lo esperaba en t = 100.
       A los 100 ms no tiene nada que reproducir.
       → Clic (click) en el audio.
El paquete 3 tardó 95 ms → llega en t = 115, tarde para t = 120.
       → se le pasó el hueco.

El PROBLEMA no es que tarde: es que lleguen en RÁFAGA.
El buffer de jitter (jitter buffer) lo resuelve: mete un
RETARDO FIJO (por ejemplo 60 ms) y entrega las muestras en
ritmo constante, reordenando y rellenando.
```

La relación con el BDP y con la cola:

```
⇐ Si hay un paquete perdido y TCP reenvía, el hueco es de un RTO
   (mínimo 200 ms en Linux) ⇒ jitter enorme.
⇐ Si un router tiene el buffer lleno, la cola cambia de largo
   de un momento a otro ⇒ jitter.
⇐ Si hay ECMP, dos paquetes del mismo flujo pueden ir por caminos
   de latencia diferente ⇒ jitter.
```

Todo el jitter viene de la `d_cola`. Y por eso **más jitter = más cola = más riesgo de bufferbloat**, y por eso los AQMs se escribieron para el jitter tanto como para la pérdida.

1.6 Congestión, pérdidas y retardo

Esta sección resuelve la **Pregunta 2 del Examen 2011-03** (6 puntos). Los tres conceptos, su definición precisa y su vínculo.

Congestión

Definición de la solución oficial, textual:

> *"La congestión está asociada a la escasez, durante un período de tiempo, de los recursos (compartidos) necesarios para procesar adecuadamente todos los paquetes que pretenden hacer uso de los mismos. Esos recursos son básicamente: capacidad de procesamiento y buffers en los routers y, enlaces."*

Los cuatro desgloses de esa definición:

1. **Es escasez de recursos COMPARTIDOS.** Si el recurso es dedicado, no hay congestión: si te reservaron 1 Mbps, tenés 1 Mbps. La congestión aparece porque **muchos usuarios comparten** el mismo enlace y los mismos buffers.
2. **Es temporal.** "Durante un período de tiempo". Una ráfaga de 200 ms no es congestión; 200 ms de saturación sostenida sí. Por eso los AQMs tienen **ventanas de tiempo**.
3. **Los recursos son de dos tipos**: capacidad de procesamiento (el fabric, la CPU del router) y buffers (los búferes de las colas de salida). Los dos se llenan.

4. **Se manifiesta en síntomas**, no es un fenómeno observable directamente. Los síntomas son **pérdidas y retardos**.

Los tres tipos de congestión, con el síntoma de cada uno:

TIPO	QUÉ SE LLENA	SÍNTOMA
Congestión de ENLACE (la más común)	la cola de salida de un puerto de un router	PÉRDIDAS (buffer lleno) + retardo de cola grande
Congestión de FABRIC (backplane)	la memoria interna del router (memoria de buffers, tablas)	Pérdidas en la entrada del router; latencia de la NIC
Congestión de CPU	la CPU del router que corre el protocolo de control	retardo de procesamiento alto; pérdida por descarte

Pérdida

Definición de la solución oficial, textual:

> *"Los paquetes se encolan en buffers en los routers antes de ser enviados por el enlace de salida; si un paquete no encuentra espacio disponible en un buffer, será descartado (pérdida)."*

Hay más causas de pérdida que el buffer lleno, y hay que conocerlas:

Causa	Dónde	Quién la detecta	¿La recupera TCP?
Buffer lleno (*tail drop*)	Cola de salida de un router	El emisor, por timeout o 3 dupACK	Sí
Checksum incorrecto	Cualquier enlace (error de medio)	El receptor de la trama (CRC)	Sí, con ARQ de capa 2
TTL = 0	Cualquier router	El router, por ICMP Time Exceeded	No: el paquete se descarta
Filtro de firewall / ACL	Cualquier router	Puede o no generar ICMP	No
Congestión de la CPU del router	Entrada del router	El emisor	Sí
Pérdida de un fragmento	Donde se perdió	El host destino, en el reensamblaje	No: se descarta **todo** el datagrama
Descarte por política de DiffServ	Cola de una clase de baja prioridad	El emisor	Sí, pero es intencional

El punto clave: una pérdida en la capa 3 es IRREVERSIBLE. IP no tiene recuperación. Si el paquete se perdió, no existe "reintentar en la red": lo único que puede hacer el emisor es **mandarlo otra vez** (que es lo que hace TCP, y por eso TCP necesita numerar bytes y reconocer).

Retardo

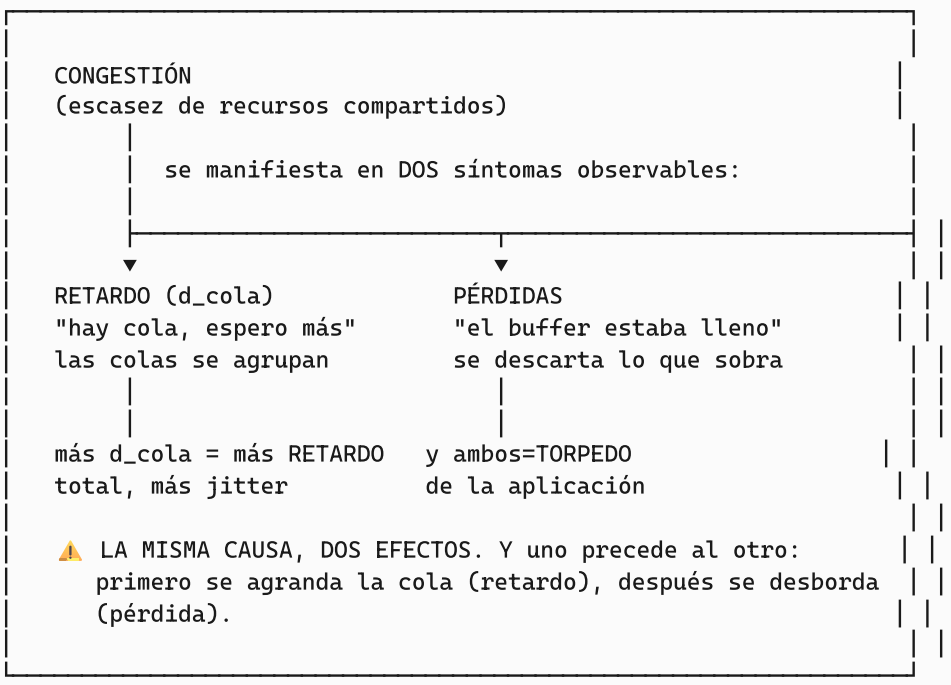
Definición de la solución oficial, textual:

> *"Si un paquete encuentra lugar en el buffer pero su envío por el enlace de salida se ve retrasado (por existir otros paquetes esperando poder hacerlo antes que él) sufrirá de retardo (retardo de cola). También puede sufrir retardo por el procesamiento que se le debe realizar en el nodo previo a ser colocado en el buffer de salida (chequeos de errores, modificaciones de campos de cabecera, ...). Adicionalmente habrá retardo debido a la propagación del paquete en el medio físico y también motivado en el ancho de banda disponible en el enlace de salida y su relación con el tamaño del paquete que se desea enviar (retardo de transmisión)."*

La oficial identifica los CUATRO componentes del retardo en un solo párrafo. Es exactamente el §1.2 de esta clase.

El vínculo entre los tres

Este es el punto que hay que_argumentar, y es lo que la pregunta pide cuando dice "explique los conceptos":



La demostración numérica del vínculo, que es lo que suma puntos:

```
Enlace de 100 Mbps, paquetes de 1500 bytes.
El enlace transmite un paquete cada 1500/10⁸ = 15 µs.

Tasa de llegada λ (paquetes/s)     | d_cola (con buffer de 40 paquetes)
──────────────────────────────────┼──────────────────────────────────────
   λ = 0      (sin tráfico)       |    0 µs
   λ = 6,67·10⁶  (= 100 Mbps)     |    0 µs   ← saturado pero sin cola
   λ = 7·10⁶                      |    1 500 × 1 = 15 µs
   λ = 8·10⁶                      |    15 000 µs = 15 ms
   λ = 9·10⁶                      |    30 000 µs = 30 ms
   λ = 10·10⁶                     |    40 × 15 µs = 600 µs  ← el ÚLTIMO cabe
   λ = 10,1·10⁶                   |    DESCARTE desde el primero
──────────────────────────────────┼──────────────────────────────────────

⚠️ A λ = 10·10⁶ = 100 Mbps, que es el máximo del enlace, la cola
   es de 600 µs. A 10,1·10⁶, que es un 0,1 % más, se pierde
   desde el primer paquete.

⇒ El retardo de cola EXPLOTA justo antes de la pérdida.
⇒ Un aumento del 0,1 % del tráfico triplica el retardo.
⇒ Por eso TCP reduce la ventana ante una pérdida: está reacting
   al síntoma de que el buffer se llenó, es decir, a que está a
   punto de desbordarse.
```

Y el corolario sobre por qué TCP no distingue: TCP ve un paquete perdido. No sabe si se perdió porque (a) se llenó el buffer de un router (congestión), (b) se corrompió en un enlace, o (c) hay un error de configuración. **Para TCP los tres son lo mismo: un paquete perdido es un paquete perdido.** Y por eso su reacción (bajar la ventana) es **conservadora en el caso (b) y correcta en el caso (a)**.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **Los cuatro componentes** del retardo, sus fórmulas y sus unidades. - `d_transmisión = L/R` y `d_propagación = d/s`, con `L` en **bits** y `d` en **metros**. - **La diferencia ancho de banda (bps) vs. velocidad de propagación (m/s)** y de qué depende cada uno. - **Que la congestión actúa sobre el retardo de encolamiento** y se manifiesta en pérdidas y retardos. - **Los valores de** `s`: ~$2{\cdot}10^8$ m/s en cobre y fibra, ~$3{\cdot}10^8$ en vacío y satélite. - **Que el retardo de cola es el único componente variable** y el único que refleja la congestión.

Cambió o se modificó:

- **La latencia de los enlaces deteriorated mucho menos de lo que la gente cree, pero los BDP crecieron muchísimo.** Un enlace continental a 100 Gbps con 100 ms de RTT tiene un **BDP de 1,25 GB**. Ningún stack de TCP con el `Window Scale` de `RFC 7323` lo llena. ⇒ **El throughput de las conexiones de larga distancia está limitado por la ventana, no por el ancho de banda**, y eso obliga a BDP-aware rate control (que es exactamente lo que hace **BBR**: no una ventana, sino un modelo de *bottleneck bandwidth* × *propagation delay*).
- `d_proc` **dejó de ser despreciable en algunos casos concretos.** En un backbone a 100/400 Gbps, la CPU del router corre **toda la pila en el plano de datos**: MPLS, ACLs, ECMP y las dos filas de QoS. El `d_proc` sube a **1 a 10 µs**, que ya es del orden del `d_transmisión` (1 µs a 10 Gbps con paquete de 1500 B). ⇒ **Decir "el procesamiento es despreciable" es correcto en una LAN y puede ser falso en un router de última generación.** Lo que sigue siendo cierto es que **no depende de la carga**, y por eso no es el componente de la congestión.

- **El control de congestión dejó de ser AIMD puro.** `RFC 9438` (CUBIC, *Standards Track*, 2023) es el default global y **no** crece linealmente. **BBRv3 nunca fue RFC** (`draft-ietf-ccwg-bbr` , *Experimental*), y es lo que se usa en QUIC y en partes de Linux.
- **La pérdida como señal de congestión se volvió opcional. ECN** (`RFC 3168`) marca en vez de descartar, y `RFC 9768` **(AccECN, 2026)** hace el feedback más preciso. **L4S (`RFC 9331` , 2023)** marca con umbral muy bajo para conseguir **latencia mínima sin descartes**: el objetivo ya no es "evitar la pérdida" sino **"evitar la cola"**. ⇒ **El jitter, que antes era un efecto secundario, pasó a ser el objetivo primario de la diseño de congestión.** Ese es el cambio conceptual más grande desde 2011.
- **El *bufferbloat* es un problema de primera clase y tiene solución en el stack por defecto.** Linux usa **FQ-CoDel** desde el kernel 3.5 (2012). Los AQMs descartan temprano para que el retardo no se dispare, aunque la pérdida "desperdicie" ancho de banda.
- **El multicast cambió la relación entre retardo y pérdida.** Con `IP_MULTICAST` los paquetes de un mismo flujo salen por interfaces con latencias distintas, y sin control de tasa la congestión de un destino **afecta a los demás** (problema de *multicast congestion collapse*, que la RFC 5771 documenta). Es un caso donde "el BDP" ya no es un número por flujo.

> **Consecuencia para el examen:** los **cuatro componentes, sus fórmulas y sus unidades no cambiaron** en 37 años. Lo que cambió son las **cifras de capacidad** y el **objetivo del control de congestión** (de "evitar la pérdida" a "evitar la cola"). Resolvé con los números del enunciado, y si hay lugar,Sumá una línea de actualización sobre ECN y AQM.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Los cuatro componentes de un paquete en dos enlaces

Datos. Un paquete de **1000 bytes** va de A a B por dos enlaces: el primero de cobre de **50 km** a **1 Mbps**, el segundo de fibra de **100 km** a **100 Mbps**. Los dos conmutadores tienen `d_proc = 0,05 ms`. No hay retardo de cola.

a) Calculá el retardo de cada componente del primer enlace. **b)** Calculá el retardo de cada componente del segundo enlace. **c)** ¿Cuál es el retardo total? ¿Qué componente domina?

Resolución, con `s = 2·10⁸ m/s` en ambos medios.

Paso 1 — Datos convertidos a las unidades correctas.

```
L = 1000 bytes × 8 bits/byte = 8000 bits      ⚠️ EN BITS
e1: R = 10⁶ bit/s,  d = 50 km = 5·10⁴ m,  s = 2·10⁸ m/s
e2: R = 10⁸ bit/s,  d = 100 km = 10⁵ m,  s = 2·10⁸ m/s
d_proc = 0,05 ms = 5·10⁻⁵ s (por conmutador, hay 1 entre los dos enlaces)
```

Paso 2 — Componentes del enlace 1 (cobre, 50 km, 1 Mbps).

```
d_proc  = 0,05 ms                       (del switch de salida)

d_cola  = 0                             (dato del enunciado: no hay cola)

d_trans = L / R = 8000 bits / 10⁶ bit/s
        = 8·10⁻³ s = 8 ms
        ⚠️ OJO: 1000/10⁶ daría 1 ms. La respuesta correcta es 8 ms
           porque L va en BITS.

d_prop  = d / s = 5·10⁴ m / 2·10⁸ m/s
        = 2,5·10⁻⁴ s = 0,25 ms
```

```
Subtotal e1 = 0,05 + 0 + 8 + 0,25 = 8,30 ms
```

Paso 3 — Componentes del enlace 2 (fibra, 100 km, 100 Mbps).

```
d_proc  = 0,05 ms

d_cola  = 0

d_trans = L / R = 8000 / 10⁸
        = 8·10⁻⁵ s = 0,08 ms
        ⚠️ 10 veces MENOS que en el enlace 1: el ancho de banda
           es 100 veces mayor, pero el retardo de transmisión
           es 8 ms / 0,08 ms = 100 veces menor... No: 8/0,08 = 100.
           Sí, 100 veces. Consistente.

d_prop  = 10⁵ / 2·10⁸ = 5·10⁻⁴ s = 0,5 ms
```

```
Subtotal e2 = 0,05 + 0 + 0,08 + 0,5 = 0,63 ms
```

Paso 4 — El retardo total.

```
d_total = Σ componentes

d_total = d_proc(e1) + d_cola(e1) + d_trans(e1) + d_prop(e1)
        + d_proc(e2) + d_cola(e2) + d_trans(e2) + d_prop(e2)

⚠️ d_proc APARECE 2 VECES: una por cada router. Si los dos
   enlaces están unidos por UN switch, d_proc(e1) es el del switch
   que recibe por el enlace 1 y manda por el 2, y d_proc(e2) es
   el del switch que recibe por el 2 y manda por el 3.
   Con SOLO 2 enlaces hay 1 switch intermedio: el host A
   transmite, un switch, y el host B recibe. Por eso d_proc
   del enlace 1 NO existe (es el host A, sin d_proc) y solo
   hay 1 d_proc.

CORRECCIÓN DEL PASO 2 y 3:
   d_proc del enlace 1 = 0 (A es un host)
   d_proc del enlace 2 = 0,05 ms (un switch)

d_total = 0 + 0 + 8 + 0,25 + 0,05 + 0 + 0,08 + 0,5
        = 8,88 ms
```

Paso 5 — ¿Qué componente domina?

```
┌──────────────┬──────────┬──────────┐
│ Componente   │ valor    │ %        │
├──────────────┼──────────┼──────────┤
│ d_trans(e1)  │ 8,00 ms  │ 90,1 %   │  ◄── DOMINA
│ d_prop(e2)   │ 0,50 ms  │  5,6 %   │
│ d_prop(e1)   │ 0,25 ms  │  2,8 %   │
│ d_trans(e2)  │ 0,08 ms  │  0,9 %   │
│ d_proc       │ 0,05 ms  │  0,6 %   │
│ d_cola       │ 0        │  0 %     │
├──────────────┼──────────┼──────────┤
│ TOTAL        │ 8,88 ms  │ 100 %    │
└──────────────┴──────────┴──────────┘
```

Interpretación — y es el punto del ejercicio:

> **El enlace lento manda en el retardo total.** El enlace de 1 Mbps tiene un `d_trans` de 8 ms; el de 100 Mbps, de 0,08 ms. El primero aporta el 90 % del retardo total **pese a recorrer la mitad de distancia**.

Y el corolario, que es la lección de infraestructura:

```
Si subimos el enlace 1 de 1 Mbps a 100 Mbps:

d_trans(e1) = 8000/10⁸ = 0,08 ms
d_total nuevo = 0,08 + 0,25 + 0,05 + 0,08 + 0,5 = 0,96 ms

MEJORA DE 8,88 ms → 0,96 ms:  9,25 VECES más rápido.

⚠️ PERO si además acortamos el enlace 2 de 100 km a 10 km:
d_prop(e2) = 10⁴/2·10⁸ = 0,05 ms
d_total = 0,08 + 0,25 + 0,05 + 0,08 + 0,05 = 0,51 ms

Ahí sí, el 1 Mbps pasa a ser el cuello de botella de verdad.
```

La respuesta a la pregunta implícita: *en una WAN, primero hay que arreglar el enlace lento (el `d_trans` grande); en una red de backbone, primero hay que acortar las distancias (el `d_prop` grande).*

⚠️ Errores típicos

- **Dividir bytes por bps.** `1000/10⁶` en vez de `8000/10⁶`. Da un factor 8 de error. **Todos los protocolos dan `L` en bytes; el retardo se calcula en bits.**
- **Poner `d_proc` tantas veces como enlaces.** Con `n` enlaces hay `n − 1` routers. Este es el error del Examen 2011-03 P5.
- **Sumar `d_trans` una sola vez.** Hay **un `d_trans` por enlace**, con el `R` de ese enlace.
- **Confundir 50 km con $5{\cdot}10^4$ m.** 1 km = 1000 m = 10^3 m. 50 km = $5{\cdot}10^4$ m.
- **Mezclar las unidades.** Si `d_trans` sale en segundos y `d_prop` en milisegundos, hay que convertir antes de sumar.

2.2 Ejercicio 2 — El BDP con la aritmética completa

Datos. Un enlace de **50 Mbps** une dos puntos separados por **2 000 km** de fibra (`s = 2·10⁸ m/s`). Se supone que no hay cola ni procesamiento.

a) ¿Cuánto tarda un paquete en llegar (latencia)? **b)** ¿Cuál es el RTT? **c)** ¿Cuál es el BDP, en bits y en bytes? **d)** Una conexión TCP tiene una ventana de **128 KB**. ¿Alcanza para llenar el enlace? ¿Cuál es el throughput máximo?

Resolución.

Paso 1 — Latencia.

```
d = 2 000 km = 2·10⁶ m
s = 2·10⁸ m/s

d_prop = 2·10⁶ / 2·10⁸ = 10⁻² s = 0,01 s = 10 ms

(ignorando d_trans y d_proc, como dice el enunciado)
latencia ≈ 10 ms
```

Paso 2 — RTT.

```
RTT = 2 × latencia = 2 × 10 ms = 20 ms = 0,02 s
```

Paso 3 — BDP.

```
R = 50 Mbps = 5·10⁷ bit/s
RTT = 0,02 s

BDP = R × RTT = 5·10⁷ bit/s × 0,02 s
    = 10⁶ bits
    = 1 000 000 bits

en bytes: 1 000 000 / 8 = 125 000 bytes = 125 kB = 122 kB aprox.
```

Interpretación: en ese enlace, **125 kB** pueden estar en tránsito al mismo tiempo.

Paso 4 — ¿Alcanza una ventana de 128 kB?

```
ventana = 128 kB = 128 × 1024 = 131 072 bytes
BDP     = 125 000 bytes

131 072 > 125 000   →  SÍ alcanza, por poco (con 4,9 % de margen)
```

Throughput máximo alcanzable:

```
Throughput = ventana / RTT
           = 131 072 bytes / 0,02 s
           = 6 553 600 B/s
           = 6,55 MB/s
           = 52,4 Mbps
```

Y el techo real del enlace:

```
R = 50 Mbps
Throughput con la ventana de 128 kB = 52,4 Mbps > 50 Mbps

⚠️ 52,4 > 50: la ventana SÍ alcanza. El tope es el ENLACE.
Throughput real = 50 Mbps = 6,25 MB/s

Para(Stream) verificar si la ventana alcanza, el criterio es:
    ventana ≥ BDP   ⟺   throughput alcanzable ≥ R
```

Paso 5 — Qué pasaría con ventanas más chicas.

ventana	= BDP?	throughput	¿aprovecha?
8 KB	8192 < 125000	8192/0,02 = 410 kB/s = 3,3 Mbps	3,3 % ⚠️
64 KB	65536 < 125000	65536/0,02 = 3,28 MB/s = 26,2 Mbps	26 % ⚠️
125 KB	= 125 000	125 000/0,02 = 6,25 MB/s = 50 Mbps	100 % ✓
1 MB	> BDP	limitado por el enlace: 50 Mbps	100 % ✓ (sobra ventana)

Interpretación y la lección:

> Con una ventana de **8 kB** (el `rwnd` por defecto de Windows durante años) en este enlace, se aprovecha el **3,3 %** del ancho de banda. El enlace no está lento: **falta ventana**. Con **64 kB**, el 26 %. Recién con **125 kB** se llena.

Y el cálculo del tiempo que tardaría en descargar 10 MB:

```
Con 8 kB de ventana:
   rounds = 10 485 760 bytes / 8192 bytes = 1280 rounds
   tiempo = 1280 × 20 ms = 25,6 s
   throughput = 10 485 760/25,6 = 409 600 B/s = 3,3 Mbps ✓

Con 125 kB de ventana (enlazada al BDP):
   rounds = 10 485 760 / 125 000 = 84 rounds
   tiempo = 84 × 20 ms = 1,68 s
   throughput = 10 485 760/1,68 = 6,24 MB/s = 50 Mbps ✓

⇒ 15,2 VECES más rápido con solo agrandar la ventana.
```

2.3 Ejercicio 3 — RTT, latencia y la diferencia entre las dos

Datos. Un `traceroute` desde Montevideo a Madrid da:

```
1  fw.campus.udelar.edu.uy (203.0.113.1)     0,412 ms  0,389 ms  0,401 ms
2  bundle-ethernet.antibot.com (198.51.100.7)  8,104 ms  8,220 ms  8,051 ms
3  ae-3.border.bb.madrid.es (192.0.2.44)      118,7 ms 119,2 ms 118,9 ms
4  www.ejemplo.es (203.0.113.200)             121,3 ms 121,0 ms 121,4 ms
```

a) ¿Cuál es el RTT a cada nodo? ¿Cuál es la latencia? **b)** ¿Cuál es el RTT mínimo de la conexión? ¿Y el jitter? **c)** ¿Cuánto del RTT es la red y cuánto es el último salto?

Resolución.

a) Lo que da el `traceroute`:

```
Cada línea muestra TRES RTT distintos, uno por cada uno de los
3 probes que manda traceroute.

El RTT es el tiempo de ida y vuelta.
La latencia (tiempo de un viaje de ida) ≈ RTT / 2
```

Salto	Nodo	RTT (v1)	RTT (v2)	RTT (v3)	**Latencia ≈ RTT/2**
1	gateway del campus	0,412	0,389	**0,389**	**0,21 ms**
2	borde del ISP	8,104	8,220	**8,051**	**4,03 ms**
3	border de Madrid	118,7	119,2	**118,9**	**59,5 ms**
4	servidor final	121,3	121,0	**121,0**	**60,5 ms**

⚠️ **La advertencia de la solución oficial del Examen 2011-03 P3, textual:**

> *"Dado que el camino que siguen los paquetes de ida y vuelta de una comunicación sobre Internet no necesariamente son el mismo, las respuestas, tanto de los nodos intermedios como del destino pueden recorrer un camino distinto al que siguieron los paquetes que las motivaron."*

⇒ **El RTT/2 no es la latencia real salvo que el camino sea simétrico.** Es una **estimación**.

b) RTT mínimo y jitter.

```
RTT mínimo de la conexión = el menor RTT del último salto
                          = 121,0 ms

Los tres RTT del último salto: 121,3 · 121,0 · 121,4
promedio = (121,3 + 121,0 + 121,4)/3 = 121,23 ms
jitter (pico a pico) = 121,4 − 121,0 = 0,4 ms
desviación estándar:
    = √(((121,3−121,23)² + (121,0−121,23)² + (121,4−121,23)²)/3)
    = √((0,0049 + 0,0529 + 0,0289)/3)
    = √0,0289 = 0,17 ms
```

Interpretación: el jitter de **0,4 ms** en un enlace transatlántico es **excelente**. Significa que la red no tiene congestión: las colas están vacías. Un jitter de 30 ms en el mismo enlace indicaría colas de 30 ms, es decir, saturación.

c) Reparto del RTT: la red vs. el último salto.

```
RTT al gateway del campus (salto 1):  0,389 ms
RTT total al destino (salto 4):      121,0 ms

RETARDO EN LA RED INTERNA (saltos 1→4):
    121,0 − 0,389 = 120,6 ms  →  99,7 % del RTT

RETARDO DEL ÚLTIMO SALTO (salto 3→4, dentro de la red de destino):
    121,0 − 118,9 = 2,1 ms     →  1,7 % del RTT
```

Lo que esto significa, y es la lección de la tabla:

> **El 99,7 % del retardo está en los enlaces de larga distancia, no en la red local.** El último tramo, que es el que el proveedor controla y por el que el usuario pregunta, aporta el 1,7 %. Por eso mejorar la conexión de un campus a un servidor remoto **rinde muchísimo menos** que acortar el camino o mejorar el enlace troncal.

Y el cálculo del ancho de banda imposible, que es el indicador definitivo de cuello de botella:

```
RTT accumulates:  la LATENCIA es irrecuperable.
Download speed:  el throughput SÍ se puede subir agrandando la ventana.

Con ventana = 65536 B (64 kB):
    throughput = 65 536 / 0,121 s = 541 620 B/s = 4,3 Mbps
    → Con un enlace de 100 Mbps se aprovecha el 4,3 %.

Con ventana = 8 MB (agregando Window Scale, RFC 7323):
    throughput = 8 388 608 / 0,121 s = 69,3 MB/s = 555 Mbps
    → Ahora sí, el enlace es el techo.
```

2.4 Ejercicio 4 — Dos paquetes de distinto tamaño en el mismo enlace

Datos. Un enlace de **10 Mbps** con fibra de **50 km** ($s = 2 \cdot 10^8$ m/s). Llegan dos paquetes: **P1 de 100 bytes** y **P2 de 10 000 bytes**.

a) Calculá los 4 componentes de cada uno. **b)** ¿Qué cambia y qué no al pasar de un paquete a otro?

Resolución.

```
P1: L = 100 bytes = 800 bits
P2: L = 10 000 bytes = 80 000 bits

R = 10 Mbps = 10⁷ bit/s
d = 50 km = 5·10⁴ m
s = 2·10⁸ m/s
```

Paso 1 — `d_trans` de cada uno.

```
d_trans(P1) = L/R = 800 / 10⁷ = 8·10⁻⁵ s = 0,08 ms

d_trans(P2) = L/R = 80 000 / 10⁷ = 8·10⁻³ s = 8 ms

⚠️ 100 veces de diferencia. El paquete grande tarda 100 veces
   más en TRANSMITIRSE. Un paquete de 100 bytes "sale" en 80 µs;
   uno de 10 000 bytes tarda 8 ms en salir entero.
```

Paso 2 — `d_prop` de cada uno.

```
d_prop(P1) = d/s = 5·10⁴ / 2·10⁸ = 2,5·10⁻⁴ s = 0,25 ms
d_prop(P2) = d/s = 5·10⁴ / 2·10⁸ = 2,5·10⁻⁴ s = 0,25 ms

⚠️ IDÉNTICOS. El paquete grande no tarda más en PROPAGARSE.
```

Paso 3 — `d_proc` y `d_cola`.

```
d_proc: los mismos 2 µs para los dos (el router no mira L
        para procesar la cabecera; el CRC sí cubre L, pero
        el CRC-32 de un hardware a 10 Gbps va a ~1 bit/ciclo)
d_cola: Supongamos que P2 llega primero y ocupa el enlace.
        d_cola(P1) = 8 ms  (espera a que P2 salga entero)
        d_cola(P2) = 0
```

Paso 4 — La tabla comparativa.

	P1 (100 B)	P2 (10 000 B)	¿cambió?
L (bits)	800	80 000	SÍ (×100)
d_proc	2 µs	2 µs	NO
d_cola	8 ms	0	SÍ
d_trans	0,08 ms	8 ms	SÍ (×100)
d_prop	0,25 ms	0,25 ms	NO ⚠️
TOTAL	8,332 ms	8,252 ms	~igual

Interpretación — el resultado que hay que entender:

> **El paquete 100 veces más grande tarda (casi) lo mismo**, y la razón es que el `d_trans` grande se **compensa** con una `d_cola` chica: el paquete grande ya estaba "en la cabeza" de la cola.

Y si los dos llegaran AL MISMO TIEMPO a una cola vacía:

```
d_cola(P1) = d_cola(P2) = 0

TOTAL(P1) = 2 µs + 0,08 + 0,25 ms = 0,332 ms
TOTAL(P2) = 2 µs + 8    + 0,25 ms = 8,252 ms

⚠️ AHORA SÍ: el grande tarda 24,8 VECES más.
```

La conclusión, que es lo que hay que decir:

```
┌──────────────────────────────────────────────────────────┐
│ El tamaño del paquete SÍ cambia el retardo total, pero solo  │
│ a través de d_transmisión (y de d_cola).                 │
│                                                          │
│ El retardo de PROPAGACIÓN es INDEPENDIENTE del tamaño.   │
│                                                          │
│ Ése es el contenido de la Pregunta 1a del Examen 2009-12:  │
│ los dos componentes que dependen de cosas DIFERENTES.    │
│                                                          │
│ Y por eso un enlace de 1 Gbps con paquete de 1500 bytes tiene │
│ d_trans = 12 µs (despreciable) mientras que uno de 100 Mbps  │
│ tiene d_trans = 120 µs (comparable a la propagación).    │
│ El ancho de banda determina si el d_trans es o no relevante. │
└──────────────────────────────────────────────────────────┘
```

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2009-12, Pregunta 1 (10 puntos) — los componentes del retardo

> **Fuente:** Examen `sirc0912` — 18 de diciembre de 2009, Pregunta 1 (10 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0912.pdf

Enunciado (texto del examen)

> Identifique los componentes del retardo en las redes de paquetes, y señale cuáles son los más significativos. En particular: **a)** Señale las diferencias entre el ancho de banda y la velocidad de propagación en el medio. **b)** ¿En qué componente del retardo influye la congestión?

Solución oficial (completa)

> Retardo de procesamiento en el nodo Por ejemplo chequeo de paridad (CRC), determinar enlace de salida (enrutamiento).
>
> Retardo de encolamiento (buffering) Espera en colas del enlace de salida para trasmisión
>
> Retardo de trasmisión Sea R = ancho de banda del enlace (bps) y L = longitud del paquete (bits), el tiempo de envío = L/R
>
> Retardo de propagación Sea d = longitud del enlace físico y s = velocidad de propagación en el medio, el retardo de propagación = d/s
>
> **a)** B es la capacidad de los puertos del dispositivo (switch o router), por ejemplo Ethernet, STM-1, mientras que s es una característica del medio, por ejemplo s ~ 2×10^8 m/seg en cobre.
>
> **b)** El retardo de encolamiento depende del nivel de congestión del router.

Análisis de la solución oficial

Lo que está muy bien:

1. **Las cuatro fórmulas están exactamente correctas y con las variables definidas.** `L/R` con `L` en bits y `R` en bps; `d/s` con `d` en longitud de enlace y `s` en m/s. Eso es exactamente lo que hay que saber, y está perfecto.
2. **Nombra los ejemplos correctos del `d_proc`**: chequeo de paridad (CRC) y "determinar enlace de salida (enrutamiento)". Los dos son las tareas reales de un router. La oficial hasta usa la palabra **"enrutamiento"** para el lookup, que es la terminología de la cátedra.

3. **La definición de `d_cola` es la buena**: "*espera en colas del enlace de salida*". Dice **del enlace de SALIDA**, que es el detalle correcto (es el enlace por el que va a salir, no por el que llegó).
4. **El apartado a) es una frase precisa**: "*B es la capacidad de los puertos del dispositivo, mientras que s es una característica del medio*". Es exactamente la distinción.

Lo que falta, y son los puntos que se ganan:

1. **No dice cuáles son "los más significativos".** El enunciado lo pide explícitamente y la oficial **no lo responde**. Hay que decir: **`d_propagación` y `d_transmisión`**, porque son los dos únicos que se pueden calcular con los datos del problema y los que dominan en enlaces de cualquier longitud relevante. `d_proc` es despreciable (microsegundos) y `d_cola` es variable (no se puede calcular).
2. **No da las unidades de `d` ni de `L`.** Dice "longitud del enlace físico" y "longitud del paquete (bits)". Falta: **`d` en METROS** y **`L` en BITS**. Es el error número uno de los alumnos.
3. **No menciona que `s` no cambia con el enlace ni con el ancho de banda.** El apartado a) lo insinúa ("característica del medio"), pero no lo dice.
4. **El apartado b) es correcto pero escueto.** Dice "el retardo de encolamiento depende del nivel de congestión del router", y **eso es todo lo que hay que decir para el punto**. Lo que suma es encadenar: congestión → cola más larga → **retardo total** ↑, y si se desborda → **pérdida**. La oficial no menciona las pérdidas.
5. **No dice que los otros tres componentes NO dependen de la congestión.** Es el complemento del apartado b) y es lo que demuestra que entendiste la estructura.

Mi respuesta completa (para los 10 puntos)

Los cuatro componentes:

```
d_end2end = d_proc + d_cola + d_transmisión + d_propagación

1. RETARDO DE PROCESAMIENTO  (d_proc)
   Qué: el tiempo que tarda el nodo en procesar el paquete entre
   que entra por una interfaz y sale por otra: verificación de
   errores (CRC o paridad), lectura de la cabecera, decremento
   del TTL y recálculo del checksum, y la búsqueda del enlace de
   salida (enrutamiento: longest prefix match).
   Depende de: la velocidad del procesador y del bus, y del ancho
   de banda del fabric.
   Típico: ~1 µs.  DESPRECIABLE en cualquier enlace de >1 km.

2. RETARDO DE ENCOLAMIENTO  (d_cola)
   Qué: la espera en la cola del enlace de SALIDA antes de que le
   toque ser transmitido.
   Depende de: el tráfico que hay por delante en la cola.
   Fórmula, para un paquete en la posición k de una cola de
   paquetes de L bits: d_cola = k · L / R_salida.
   Puede ser CERO (si el enlace está libre) y NO TIENE COTA
   SUPERIOR.  Es EL ÚNICO COMPONENTE VARIABLE.

3. RETARDO DE TRANSMISIÓN  (d_transmisión)
   Qué: el tiempo que tarda el emisor en poner el paquete entero
   en el cable, del primer bit al último.
   Fórmula: d_trans = L / R
     L = longitud del paquete EN BITS
     R = velocidad de transmisión del enlace (bps), es decir, su
         ancho de banda.
   Depende de: L y R.  NO del medio, NO de la distancia.
   ⚠️ Es el componente que cambia al cambiar el tamaño del paquete.

4. RETARDO DE PROPAGACIÓN  (d_propagación)
   Qué: el tiempo que tarda la SEÑAL en recorrer el medio físico.
   Fórmula: d_prop = d / s
     d = longitud del enlace físico, EN METROS
     s = velocidad de propagación en el medio, EN m/s
         (≈2·10⁸ m/s en cobre y fibra; ≈3·10⁸ en vacío)
   Depende de: d y s.  NO del paquete, NO del ancho de banda.
   ⚠️ Es el componente que NO cambia al cambiar el tamaño del
      paquete.

_____________________________________________________________

CUÁLES SON LOS MÁS SIGNIFICATIVOS

Los dos SIGNIFICATIVOS son d_transmisión y d_propagación:
 · Son los únicos dos que se pueden CALCULAR con los datos del
   enunciado (L, R, d, s).
 · Son los que DOMINAN: en un enlace de 100 km a 1 Gbps con un
   paquete de 1500 bytes, d_trans = 12 µs y d_prop = 500 µs, y
   los otros dos son despreciables frente a eso.

d_proc es despreciable: del orden de 1 µs, contra los ms de
propagación de cualquier enlace interurbano.
d_cola es el más relevante OPERATIVAMENTE (es el que la
congestión mueve), pero NO es calculable a priori.
```

a) Las diferencias, con las unidades:

ANCHO DE BANDA (B o R) - Unidad: bits por SEGUNDO (bps) - Qué es: la CAPACIDAD MÁXIMA de transferencia del enlace - De qué depende: del EQUIPO (los puertos del switch o router, el transceiver, la modulación): Ethernet de 1 Gbps, STM-1 de 2,048 Mbps, 100 GbE. - NO depende de: el medio físico ni de la longitud del enlace - Aparece en: d_transmisión = L / R

VELOCIDAD DE PROPAGACIÓN (s) - Unidad: METROS por SEGUNDO (m/s) - Qué es: la rapidez con que viaja la SEÑAL por el medio - De qué depende: del MEDIO FÍSICO (la física del material): cobre $\approx 2 \cdot 10^8$ m/s, fibra $\approx 2 \cdot 10^8$ m/s, vacío $3 \cdot 10^8$ m/s. - NO depende de: el equipo, el ancho de banda ni la longitud - Aparece en: d_propagación = d / s

CONCLUSIÓN: son magnitudes de UNIDADES DISTINTAS (bps frente a m/s) que dependen de COSAS DISTINTAS (el equipo frente al medio). Duplicar el ancho de banda reduce d_transmisión a la mitad y NO cambia d_propagación. Un enlace de 1 Gbps no es 50 veces más rápido que uno de 100 Mbps: es 50 veces más ANCHO.

```
**b) En qué componente influye la congestión:**
```

La congestión influye SÓLO en el RETARDO DE ENCOLAMIENTO (d_cola).

Porque es el único componente que depende del estado de la cola, y la congestión ES el estado de la cola. Los otros tres dependen solo de L, R, d, s y del procesador, que no cambian con la carga.

La cadena completa: más tráfico ofrecido → las colas se agrandan → d_cola AUMENTA → el retardo total AUMENTA → y si el buffer se DESBORDA, el paquete se descarta → PÉRDIDAS

Es decir: la congestión se MANIFIESTA en retardo de cola y en pérdidas, que son los dos únicos síntomas observables. Nunca modifica d_transmisión, d_propagación ni d_proc.

```
## 3.2 ⭐ Examen 2011-03, Pregunta 2 (6 puntos) — congestión, pérdidas y retardo

> **Fuente:** Examen `sirc1103` — 21 de febrero de 2011, Pregunta 2 (6 puntos).
> **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

### Enunciado (texto del examen)

> Dada una red de conmutación de paquetes, explique los conceptos de congestión, pérdidas y re

### Solución oficial (completa)

> **Congestión:** La congestión está asociada a la escasez, durante un período de tiempo, de l
>
> **Pérdida:** Los paquetes se encolan en buffers en los routers antes de ser enviados por el
>
> **Retardo:** Si un paquete encuentra lugar en el buffer pero su envío por el enlace de salid

### Análisis de la solución oficial

**Lo que está muy bien:**

1. **La definición de congestión tiene los cuatro elementos correctos**: escasez, recursos com

2. **Dice explícitamente que la congestión "se manifestará en pérdidas y/o retardos"**, y que

3. **La definición de pérdida da el mecanismo exacto**: "los paquetes se encolan en buffers an

4. **La definición de retardo menciona los cuatro componentes en un párrafo**, con los ejemplo

**Lo que falta o se puede mejorar:**

1. **No da las fórmulas.** La oficial describe los componentes pero no escribe `L/R` ni `d/s`.
2. **No dice que la congestión "no existe" sin compartición.** Dice "recursos compartidos", qu
3. **La definición de retardo no distingue el retardo "de congestion" del "de física".** Podrí
4. **No menciona que una pérdida en la capa 3 es irreversible.** Un paquete descartado en un r
5. **No menciona que hay pérdidas por otras causas** (checksum, TTL, filtros), que es lo que i

### Mi respuesta completa (para los 6 puntos)

**Congestión.** Es la **escasez, durante un período de tiempo, de los recursos compartidos** n

Tres precisiones que hay que hacer:

- **Es un fenómeno de compartición.** Si el recurso fuera dedicado, no habría congestión. La c
- **Es temporal.** Una ráfaga de 200 ms no es congestión; 200 ms de saturación sostenida sí. P
- **No es observable directamente.** La congestión es un estado interno de la red; lo que se o

**Pérdida.** Un paquete **se descarta cuando no encuentra lugar en un buffer**. Los paquetes s

Hay que agregar dos precisiones:

- **Una pérdida de capa 3 es irreversible.** El paquete descartado no vuelve: **IP no tiene me
- **No toda pérdida es por congestión.** También hay checksum incorrecto (error del medio), TT

**Retardo.** Es el tiempo que tarda un paquete en ir de un nodo al siguiente, y tiene cuatro c

**El vínculo entre los tres, que es lo que hay que argumentar:**
```

La congestión es la CAUSA; el retardo y la pérdida son los SÍNTOMAS. Y los dos síntomas vienen del mismo mecanismo: la cola.

┌──────────────

```
**Interpretación y el punto del ejercicio:** acá **el `d_trans` DOMINA** (86 % del total), a d

El criterio: el `d_trans` importa cuando el enlace es **lento** o el paquete es **grande**. El

</details>

**2.** ¿Por qué la solución oficial del Examen 2009-12 no responde la parte de "señale cuáles

<details><summary>Respuesta</summary>

Porque es un detalle que la cátedra da por obvio, pero **la pregunta lo pide explícitamente**

**La respuesta correcta: los más significativos son `d_transmisión` y `d_propagación`.**

Las razones son dos:

1. **Son los únicos calculables** con los datos que da el enunciado. `d_proc` depende del mode
2. **Son los que dominan en términos absolutos**: `d_prop` va de milisegundos a cientos de mil

`d_cola` es el más relevante **operativamente** — es el que la congestión mueve y el que produ

</details>

**3.** Dos paquetes del mismo tamaño viajan por el mismo enlace. Uno va por cobre y el otro po

<details><summary>Respuesta</summary>

**Solo cambia la `d_propagación`, y muy poco.**
```

Cobre: $s = 2{,}00 \cdot 10^8$ m/s Fibra: $s = 2{,}04 \cdot 10^8$ m/s $d = 1000$ km $= 10^6$ m

$d_{prop}(\text{cobre}) = 10^6 / 2{,}00 \cdot 10^8 = 5{,}000$ ms $d_{prop}(\text{fibra}) = 10^6 / 2{,}04 \cdot 10^8 = 4{,}902$ ms

diferencia = 0,098 ms = 98 µs (un 2 %)

```
**Lo que NO cambia:**

- `d_transmisión = L/R`: el medio no entra en la fórmula. Si el enlace es de 1 Gbps por cobre
- `d_proc`: depende del equipo, no del medio.
- `d_cola`: depende de la carga.

**La lección: la fibra no se compra por latencia.** Se compra por atenuación (26 dB/km contra

El mismo argumento del Ejercicio 2.2 de la Clase 1, y el mismo número: **~0,1 ms de diferencia

</details>

**4.** ¿Qué es el BDP y por qué importa? ¿Es un concepto nuevo o es antiguo?

<details><summary>Respuesta</summary>

**BDP** = *Bandwidth-Delay Product* = `R × RTT`, en bits. Es **la cantidad de datos que pueden

**No es nuevo.** Aparece en la literatura deinale 1980s (el modelo de "ventana de secado" de l

**Por qué importa:**
```

Throughput máximo alcanzable = min(R , ventana / RTT)

```
Es decir: o estás limitado por el **ancho de banda del enlace**, o estás limitado por tu **ven

**Ejemplo:** enlace de 100 Mbps, RTT de 40 ms. BDP = `10⁸ × 0,04 = 4·10⁶ bits = 500 kB`. Con u

**La consecuencia de diseño:** `Window Scale` (`RFC 7323`, 2014) es lo que hace posible aprove

**Y el ejemplo extremo del BDP:** un enlace de 10 Gbps de un operador, con 10 000 flujos TCP a

</details>

**5.** ¿Por qué el jitter rompe el audio pero no rompe la descarga de un archivo?

<details><summary>Respuesta</summary>

Porque **la descarga de un archivo no tiene ritmo**, y el audio sí.

**Descarga de archivo:**
```

Retardo 1 = 400 ms → el usuario espera 400 ms Retardo 2 = 80 ms → el usuario espera 80 ms No hay diferencia perceptible. El archivo llega entero igual. El retardo total importa, y 80 vs 400 ms es 1,6 % de la descarga.

```
**Audio (un códec que reproduce cada 20 ms):**
```

t=0 t=20 t=40 t=60 t=80 t=100 t=120 │ │ │ │ │ │ │ │P1─────────┤ │ │ │ │ │ llega en t=80, perfecto │ │P2─────────┤ │ │ │ │ llega en t=110, tarde │ │ │P3─────────┤ │ │ │ llega en t=100, tarde │ │ │ │P4─────────┤ │ │ llega en t=138, tardísimo │ │ │ │ │ │ │ audio: ▓▓▓

```
| El emisor lo mide como latencia grande. | | | | | └──▶ cuando la cola llena el buffer: DESCARTE. | |
El emisor lo mide como pérdida. | | | | Son los DOS síntomas del mismo fenómeno, y el retardo es el |
|                                  symptomático                                  PRECOZ.                                  |
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

```
**La demostración que hay que saber de memoria:** si el buffer tiene `n` paquetes y la tasa de
```

L/R = 1500/10^8 = 15 µs por paquete diferencia entre el primero y el último = 1000 × 15 µs = 15 ms

```
Y **el paquete que llega justo cuando el buffer se llena tiene `d_cola = 15 ms`; el que llega

</details>

**7.** En un enlace de 1 Gbps con 3000 paquetes en cola, ¿cuánto tarda en salir el último?

<details><summary>Respuesta</summary>
```

R = 10^9 bit/s L = 1500 bytes = 12 000 bits

d_cola(último) = k × L/R = 3000 × 12 000 / 10^9 = 36 000 000 / 10^9 = 0,036 s = 36 ms

Y d_trans (su propia transmisión) = 12 000/10^9 = 12 µs = 0,012 ms

d_total del último = 36 + 0,012 = 36,012 ms

```
**Interpretación:** 36 ms de cola en un enlace de 1 Gbps. Un `ping` a un host a un salto de di

**Y si el buffer fuera de 3 GB:** `3000 × 1000 = 3 000 000 paquetes × 15 µs = 45 segundos`. Un

</details>

**8.** Explique la diferencia entre latencia y RTT, y por qué `traceroute` puede mentir sobre

<details><summary>Respuesta</summary>

**Latencia** = el tiempo de **un viaje de ida**. **RTT** = el tiempo de **ida y vuelta**, que
```

RTT = d_ida/s + cola_ida + d_vuelta/s + cola_vuelta + proc latencia ≈ RTT/2 ⚠️ SOLO si d_ida = d_vuelta

```
**Por qué `traceroute` miente, textual de la solución oficial del Examen 2011-03 P3:**

> *"Dado que el camino que siguen los paquetes de ida y vuelta de una comunicación sobre Inter

**El caso clásico:** el tráfico de ida pasa por un enlace de fibra de 2 ms y el de vuelta por
```

latencia real de ida = 2 ms RTT = 2 + 600 = 602 ms latencia estimada = 602/2 = 301 ms ⚠️ contra 2 ms real ```

El error es de un factor 150. Y por eso `traceroute` **no sirve para medir latencia**: sirve para **descubrir el camino de ida**.

Y la otra trampa del `traceroute`, que la oficial también menciona:

> *"pueden estar filtradas (por razones de seguridad y/o performance) [...] por lo que el no obtener respuesta desde los nodos intermedios o desde el nodo final no nos permite concluir que haya algún problema en la red o en el destino."*

Un asterisco (`*`) en la salida de `traceroute` significa **"no obtuve respuesta"**, que puede ser: el nodo no respondió, el camino de vuelta es otro, o **el router filtró el ICMP de vuelta**. **No significa que el nodo esté caído.**

🔗 Referencias

Exámenes - Examen `sirc0912` (18-dic-2009) — **P1 (10 pts): componentes del retardo, ancho de banda vs. velocidad de propagación, congestión** (esta clase) - Examen `sirc1103` (21-feb-2011) — **P2 (6 pts): congestión, pérdidas y retardo** · **P5 (10 pts): retardo end-to-end con 3 enlaces y 2 conmutadores** → **Clase 6** - Examen `sirc1103` — P3 (6 pts): `traceroute`, los dos sabores (UDP e ICMP) y por qué el camino de vuelta puede ser otro - Examen `sirc1002` (26-feb-2010) — P5 (10 pts): jitter vs. retardo - Examen `sirc0903` (20-feb-2009) — P5 (6 pts): full duplex vs. half duplex - Examen `sirc0612` (26-dic-2006) — P1 (5 pts): modelo OSI y capa física

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., **cap. 1 §1.5** "El retardo en las redes de paquetes" (pág. ~26 a ~34): los cuatro componentes con el diagrama del router y los números - Tanenbaum & Wetherall, **cap. 1 §1.2** "Tipos de red" — la comparación LAN/WAN con los números de latencia - Tanenbaum & Wetherall, **cap. 1 §1.3** — store-and-forward y conmutación de circuitos frente a paquetes - Kurose & Ross, *Computer Networking*, cap. 1 §1.7 "The Network Core: Delay and Loss" — la derivación de `L/R` y `d/s` con los cuatro Diagramas

RFCs - `RFC 9293` — *TCP* (2022, *Internet Standard*): el campo *window* y el control de flujo que **es** una manera de administrar el BDP - `RFC 7323` — *TCP Extensions for High Performance* (2014): ***Window Scale***, sin el cual el throughput de larga distancia está limitado a ~20 Mbps - `RFC 5681` — *TCP Congestion Control* (2009): los cuatro algoritmos que reaccionan a la **pérdida** como señal de congestión - `RFC 9438` — *CUBIC* (2023, *Standards Track*): el default global, que ya no es AIMD puro - `RFC 3168` — *ECN* (2001): marcar en vez de descartar · `RFC 9768` **AccECN** (2026) · `RFC 9331` L4S (2023) - `RFC 5437` — *Packet Scheduling and Dropping* (2009): los AQMs que controlan el retardo de cola (CoDel, RED, PIE)

Clase siguiente: 6 — Taller de retardos (todo resuelto)